

2026-08-09-Newsletter_Blocchi_design

Newsletter a blocchi — design

Creato: 2026-08-09 · **Aggiornato:** 2026-08-09 (decisioni prese, §6) · **Stato:** APPROVATO **Origine:** collaudo newsletter del 2026-08-09 (RISULTATI TEST NEWSLETTER 9 Agosto 2026.pdf)

1. Il problema

La newsletter che SFT manda davvero — quella del vecchio CMS Drupal allegata al collaudo — è un **documento composto**: immagine di testata con titolo, blocchi di testo, riquadri tour con copertina e pulsante “Vai alla pagina del Tour”, blocco informazioni (chi può partecipare, costi, adesioni), footer societario.

Quello che abbiamo costruito è **una casella di testo formattato** dentro un template fisso. Non è la stessa cosa, e la differenza non si colma aggiungendo un pulsante.

Cosa manca, in concreto

	Stato
Immagini nel corpo	Impossibili : la toolbar Quill non ha il pulsante immagine (NewsletterPage.razor:53-59)
Logo	C'è, ma incorporato come data: URI → Outlook per Windows non lo renderizza
Copertine dei tour	Non riutilizzabili: la galleria produce WebP , che Outlook non supporta
Bozza	stato='bozza' è già ammesso dal CHECK e non lo scrive nessuno
Anteprima	Assente
Clonazione	Assente — ma le newsletter sono quasi sempre varianti della precedente
Invio reale a un solo indirizzo	Assente (l’“invio di prova” forza [TEST] e l’italiano)
CompanyEmailTemplate	Classe statica C# con l’HTML inline: invisibile e non modificabile dall’utente

Cosa invece c'è già e va riusato

- **Supabase Storage + IWebMediaStorage.BuildPublicUrl** → URL pubblici https, che è esattamente ciò che vogliono le immagini nelle email.
 - **I dati dei tour**: copertina, titolo, slug. Il grosso di una newsletter SFT è “ecco i nuovi tour, clicca qui” — dati che il gestionale ha già e che oggi andrebbero ricopiati a mano.
 - **web_traduzioni** polimorfica (entita + entita_id) e il motore Claude.
 - **fn_web_tour_contenuti_clona** come modello per la clonazione.
-

2. Decisioni di progetto

2.1 — Blocchi in tabella, non in JSONB

web_newsletter_blocchi, figlia di web_newsletter_invii, sul modello di web_tour_itinerario.

Perché non JSONB: le traduzioni. web_traduzioni indirizza (entita, entita_id, campo, lingua) e ha bisogno di **identificatori stabili** per ogni testo traducibile. Con un JSONB si dovrebbe inventare un indirizzamento per posizione, che si rompe al primo riordino dei blocchi. Con una tabella ogni blocco ha una PK e la traduzione ci si aggancia come già fa per le giornate dell'itinerario. In più il riordino ha già un pattern collaudato (fn_web_tour_itinerario_reorder).

2.2 — Tipi di blocco

Derivati **dalla newsletter reale**, non inventati:

Tipo	Contenuto	Traducibile
testata	immagine di sfondo + titolo + sottotitolo	titolo, sottotitolo
testo	HTML ricco (Quill senza immagini)	corpo
info	icona a fianco + titolo + testo	titolo, sottotitolo, corpo
tour	riferimento a un'edizione → copertina, titolo, testo, link	testo introduttivo, etichetta pulsante
immagine	immagine + testo alternativo + link opzionale	alt
pulsante	etichetta + URL	etichetta
separatore	spazio o linea	—

Aggiunto il 2026-08-11: info. Emerso confrontando la newsletter reale, dove ricorre tre volte (CHI PUÒ PARTECIPARE, COSTI, ADESIONI). Non era coperto: il riquadro tour è legato a un'edizione e porta un pulsante, immagine

occupa tutta la larghezza e il testo finirebbe sotto invece che a fianco. Qui l'immagine è un' **icona di accompagnamento** (~110px), non una copertina, e non c'è collegamento. La colonna dell'icona resta larga uguale anche quando l'icona manca: altrimenti un riquadro senza icona sfalserebbe i titoli rispetto agli altri incolonnati.

⚠ **Le immagini non devono contenere testo.** Il testo dentro un'immagine non viene tradotto e resterebbe in italiano nelle newsletter agli stranieri. L'avviso è nel dialogo, accanto alla scelta dell'immagine, e vale per tutti i blocchi con immagine — non solo per info.

Rivisto il 2026-08-09 (§6.3, §6.5): intestazione e footer **sono blocchi**, ma di tipo **obbligatorio** — non eliminabili. Il footer è inoltre **componibile per azienda** (quali campi, ordine, colonne), con il link di disiscrizione iniettato dal sistema e non rimovibile. Ogni blocco ha inoltre proprietà di **layout** (sinistra|destra|pieno, colonne 1|2).

2.3 — Il blocco tour si compila da solo

L'operatore sceglie un' **edizione** (viaggio + data); il sistema prende copertina e titolo e costruisce il link dallo slug. È il blocco che fa risparmiare più tempo.

Tutti i testi restano modificabili (titolo, periodo, testo introduttivo): quelli del tour sono un **punto di partenza**, non un vincolo — una newsletter ha un tono diverso da una scheda prodotto, e il titolo che funziona sul sito non è detto funzioni in una email. Le modifiche vivono nella newsletter e **non toccano la scheda del viaggio**.

Conseguenza sul comportamento (2026-08-09): se si ri-sceglie il tour su un riquadro che contiene già dei testi, il sistema **chiede** se sostituirli. Rispondendo *No* aggiorna solo immagine e collegamento. Sostituire in silenzio il titolo appena riscritto dall'operatore è una perdita che non lascia tracce e che si scopre a newsletter spedita.

2.4 — Immagini compatibili con l'email

Regola: **niente data: URI, niente WebP**. Ogni immagine usata in una newsletter deve essere raggiungibile via URL pubblico https e in formato **JPEG o PNG**.

Conseguenze: - serve una **resa JPEG** delle immagini usate nella newsletter. Le copertine dei tour sono WebP: al momento dell'inserimento nel blocco si genera (una volta) un derivato JPEG in Storage sotto newsletter/, e si usa quello; - **il logo passa da base64 a URL pubblico**, con caricamento in Storage. È un cambio a CompanyEmailTemplate che sistema anche il logo delle email non-newsletter, oggi probabilmente invisibile in Outlook; - l'HTML generato è **tabellare, larghezza fissa 600px, stili inline**. Niente flex, niente grid, niente <style> in <head>: sono le regole che Outlook impone.

Eccezione: le icone restano PNG (2026-08-10). La regola “tutto in JPEG” vale per le fotografie, non per le icone: il JPEG non ha canale alfa e appiattisce la trasparenza su bianco. Verificato generando un PNG trasparente e passandolo nella pipeline — gli angoli uscivano #FFFFFF opachi, che su un pulsante social colorato significa un **riquadro bianco attorno al logo**. Il PNG è supportato da tutti i client di posta, Outlook compreso, quindi non c'è nulla da guadagnare a convertirlo. ToEmailIconPngAsync ridimensiona a

128px e conserva l'alfa; il caricamento accetta **solo PNG**, con il controllo lato applicazione e non solo nell'attributo Accept del browser, che è un suggerimento aggirabile.

2.5 — Bozza: si finisce ciò che è già previsto

La riga di `web_newsletter_invii` viene creata **quando si crea la newsletter**, con `stato='bozza'`, non al momento dell'invio. Da lì: si modifica, si vede in anteprima, si clona, si invia. `corpo_html` conserva l'**HTML renderizzato al momento dell'invio**, come istantanea di ciò che è davvero partito (serve allo storico, non alla modifica).

2.6 — Traduzione per campo, non del blob

Oggi l'intero HTML passa dentro Claude. Con i blocchi si traducono **solo i campi di testo**: meno token, e soprattutto le immagini e la struttura non possono più tornare indietro rovinare.

3. Cosa cambia per l'utente

1. **Newsletter** diventa un elenco: bozze e inviate, con *Nuova, Duplica, Modifica, Anteprima, Invia*.
2. La composizione è un elenco di blocchi con aggiungi / riordina / elimina.
3. **Anteprima** mostra la newsletter come la vedrà il destinatario, nella lingua scelta.
4. **“Invia a me”** manda il rendering **reale** a un solo indirizzo: nessun [TEST], lingua a scelta. *(La lingua a scelta è arrivata dopo, il 2026-08-18: fino ad allora la prova era sempre italiana, e il collaudo delle traduzioni costava un invio a tutti. Tendina accanto all'indirizzo.)*
5. **Duplica** crea una bozza identica da modificare — il caso più frequente.

4. Piano di lavoro

Fase	Contenuto	Dipendenze
1. DB 	<code>web_newsletter_blocchi</code> + CRUD + reorder + <code>fn_web_newsletter_clona</code> ; bozza — FATTA (script 512, 2026-08-09)	—
2. Rendering 	<code>NewsletterHtmlRenderer</code> (blocchi → HTML tabellare); resa JPEG; logo via URL — FATTA (2026-08-09)	1
3. UI 	elenco newsletter, composizione a blocchi, anteprima, duplica, selettore edizione del blocco tour — FATTA (2026-08-09). Il selettore riusa il componente condiviso <code>TravelDataSelectorDialog</code> , reso generico con <code>TitoloPersonalizzato/EtichettaConferma</code> invece di duplicarlo	1, 2

Fase	Contenuto	Dipendenze
4. Traduzioni	per campo invece che sul blob	1, 3
5. Invio	“invia a me” reale, invio campagna sul rendering	2, 3

Ogni fase si chiude con build verde e le righe di test corrispondenti nel Piano di Test §8.

Nota sulla Fase 2 — come è stata verificata

`NewsletterHtmlRenderer` è deliberatamente una classe **pura**: nessuna dipendenza, nessun I/O, riceve record semplici e restituisce una stringa. Non è solo pulizia — è ciò che l’ha resa **verificabile**, visto che i test unitari non girano da riga di comando in questo progetto (ProjectReference al progetto principale, solo maccatalyst → NU1201).

Il renderer è stato compilato in un progetto console a parte e fatto girare su una newsletter di esempio ricalcata su quella reale di SFT (testata, testo, un tour con immagine a sinistra, due tour affiancati, separatore, pulsante, footer). Controlli automatici sull’HTML prodotto:

Verifica	Esito
<code>display:flex / display:grid</code> assenti	ok
nessuna URI data:	ok
nessun <code>.webp</code>	ok
nessun <code><style></code> nell’head	ok
tutte le <code></code> hanno l’attributo <code>width</code>	5 su 5
link di disiscrizione presente	ok
larghezza colonne affiancate	$268+268 = 536$, esatta dentro i 600px con i margini

Campione dell’HTML prodotto: `scratchpad/anteprima_newsletter_esempio.html`.

Scelta di resa da conoscere: nel blocco testata il titolo va **sotto** l’immagine, non sovrapposto. Il testo sopra un’immagine richiede `background-image`, che in Outlook funziona solo con VML: si è preferita una resa uguale ovunque a una che si rompe su un client.

I pulsanti sono **a tabella** e non `<a>` stilizzati: Outlook ignora `padding` e `background` su un link e il risultato sarebbe testo blu sottolineato al posto del bottone.

5. Punti confermati (2026-08-09)

- Anteprima solo nell’applicazione.** Niente “vedi nel browser”: richiederebbe una pagina pubblica, quindi la Fase 3 del sito. In più c’è l’invio di prova a un indirizzo scelto.
- Nessuna importazione da Drupal.** Si riparte da zero: si compone la prima a mano e da lì si clona. Questo rende la clonazione e i modelli (§6.2) non un comodo ma il meccanismo principale.

3. “**Invia a me**” diventa “**invia di prova a un indirizzo scelto**”: l’utente scrive l’indirizzo, non si prende automaticamente quello aziendale. Antonio può volerla su una casella personale, o mandarla al grafico.
-

6. Decisioni sulla componibilità (2026-08-09)

Discussione su quanto debba essere “lego” il sistema. Sintesi di cosa si fa e cosa no.

6.1 — Comporre l’ordine: già previsto

I blocchi stanno in tabella con **ordine**: qualsiasi sequenza, qualsiasi numero, qualsiasi ripetizione (tre tour, testo, altri due tour, immagine). Non era una limitazione da rimuovere.

6.2 — Modelli con nome: SÌ

Una struttura riutilizzabile è una newsletter con `is_modello = true`: non compare nello storico, si duplica invece di inviarsi. Costa un booleano e un filtro, **perché la clonazione fa già il lavoro vero**. Con l’importazione da Drupal esclusa, è la via con cui si costruisce il patrimonio di partenza.

6.3 — Blocchi obbligatori e facoltativi: SÌ

Proprietà del **tipo** di blocco, non della singola istanza: intestazione e footer sono obbligatori e non eliminabili, gli altri liberi. Serve a impedire che spariscano per un clic distratto.

6.4 — Varianti di disposizione: SÌ, come proprietà

Il bisogno di “posizione fisica uno rispetto all’altro” si copre con proprietà del blocco (layout, colonne), non con tipi nuovi: immagine a sinistra col testo a destra, invertiti, a piena larghezza, due tour affiancati invece che impilati.

Attenzione al significato, che dipende dal tipo di blocco (emerso in collaudo il 2026-08-10):

Blocco	layout	colonne = 2
tour, immagine	posizione dell’immagine (sinistra/destra/pieno)	due tour affiancati
testo	allineamento (sinistra/centro/destra)	—
pulsante	allineamento nella riga — ignorato quando è in fila	in fila coi pulsanti vicini

Il caso che l’ha reso evidente: tre pulsanti allineati sinistra/centro/destra apparivano **a scaletta**, non in fila. Era corretto — ogni blocco occupa una riga a piena larghezza e l’allineamento lo posiziona dentro quella riga — ma nessuno se lo aspetta: una fila di

pulsanti social è una cosa che ogni newsletter ha. Da qui colonne = 2 sui pulsanti, che raccoglie **tutta la sequenza** di pulsanti consecutivi così marcati in un'unica riga divisa in celle uguali (tre pulsanti → tre celle da 33,33%), non solo una coppia come per i tour.

La fila è fatta di tre caselle: sinistra, centro, destra (rivisto il 2026-08-10). Ogni pulsante **dichiara quale occupare** tramite il proprio layout, e ogni casella ospita un solo pulsante.

Le due versioni precedenti erano entrambe sbagliate, in modi opposti: 1. l'allineamento valeva *dentro* la cella → ogni pulsante finiva in un punto diverso del suo terzo e la fila usciva sbilanciata; 2. l'allineamento veniva *ignorato* e i pulsanti si disponevano nell'ordine dei blocchi → la posizione risultava decisa dall'ordine in elenco, che in una lista **verticale** nessuno legge come “sinistra-centro-destra”. Per l'utente era una disposizione arbitraria.

Da qui il **massimo di tre**: le posizioni dichiarabili sono tre. Con più pulsanti servirebbe esprimere *chi sta a destra di chi*, che è un modello diverso e molto più oneroso. Un quarto pulsante consecutivo apre una **nuova fila** invece di stringere gli altri.

Le caselle vuote **restano vuote**: sinistra + destra senza il centro è una disposizione legittima, e riempire il buco spostando i pulsanti tradirebbe la scelta fatta.

Il conflitto (due pulsanti sulla stessa posizione, o più di tre in fila) è impedito al **salvataggio del blocco** e non solo prima dell'invio: un pulsante è valido da solo e diventa un problema per via dei vicini, quindi la verifica guarda l'insieme risultante (NewsletterBloccoValidator.ValidaFilePulsanti).

Larghezze in **percentuale** (33,33%) e non in pixel: con tre celle la divisione in pixel lascia un resto e l'ultima colonna risulta più stretta.

6.5 — Footer componibile per azienda: SÌ, con un vincolo

Quali campi (ragione sociale, indirizzo, P.IVA, email, telefono, sito, social), in quale ordine, su quante colonne, con quale allineamento. La configurazione vive in `web_aziende_funzioni.parametri` (JSONB), che è già usato così per la config recensioni (script 481).

 **Il link di disiscrizione non è componibile.** Viene iniettato dal sistema e non è rimovibile. È un obbligo di legge per la posta commerciale ed è l'unico meccanismo che rende possibile la disiscrizione. Un'azienda che se lo togliesse per una scelta grafica manderebbe newsletter non conformi a tutta la lista.

6.6 — Tipi di blocco definibili dall'utente: NO

Valutata e **scartata**. Non per l'effort in sé, ma per tre conseguenze:

- **Le traduzioni perdono l'ancoraggio.** `web_traduzioni` indirizza (entita, entita_id, campo, lingua). Con campi definibili dall'utente, campo diventa una stringa che si può rinominare o cancellare: il giorno che si modifica un tipo, le traduzioni delle newsletter che lo usano puntano al vuoto, in silenzio e in quattro lingue.
- **Il tipo è mutabile, la newsletter inviata no.** Cambiare un tipo dopo dieci invii obbliga o a versionare i tipi, o ad accettare che l'anteprima di una newsletter vecchia

mostri qualcosa di diverso da ciò che è partito.

- **Riporta il problema di partenza.** Definire la resa di un blocco significa scrivere HTML per email, con le regole di Outlook che nessun utente conosce: è esattamente ciò da cui la strada a blocchi doveva proteggere.

In più i tipi sarebbero globali (violando l'invariante silos) o per-azienda (moltiplicando la manutenzione per il numero di clienti).

6.7 — Blocco html libero: NO

Era stato proposto come valvola di sfogo. **Escluso a priori** su indicazione del committente, con una motivazione più forte di quella tecnica: nessun utente normale saprebbe usarlo, e un HTML malformato non peggiora solo la resa — può **far fallire l'invio**. Se in futuro arriverà una richiesta concreta che le varianti di layout non coprono, sarà una valutazione a sé.

6.8 — Effort

Aggiunta	Impatto sul piano di §4
Modelli con nome	trascurabile
Obbligatorio / facoltativo	trascurabile
Varianti di layout	piccolo (rami nel renderer)
Footer componibile per azienda	medio (schermata di configurazione + renderer)

Nel complesso il lavoro cresce di circa **metà** rispetto al piano iniziale. I tipi definibili dall'utente, se fossero stati accolti, non si sarebbero sommati ma **moltiplicati**: da “finiamo la newsletter” a “costruiamo un page-builder per email”.